FPGA Practical Tutorial

ETree FCore2S — English Translation

Original document: Version 1.4 · 9 October 2022

Translation notes

This English edition translates the Chinese explanatory text, headings, revision history, table of contents and instructional captions that could be extracted from the 128-page source PDF. FPGA/Verilog identifiers, device part numbers, register names, hexadecimal values, command names and software menu labels are kept unchanged. The source PDF contains many screenshots, schematics and code listings; screenshot graphics and labels embedded in images are not redrawn or translated here. Use the original PDF alongside this English text when following visual steps or checking the complete HDL listings.

Some statements and code comments in the source appear inconsistent or contain typographical errors. They have been translated faithfully rather than silently corrected.

ETree · FCore2S FPGA Practical Tutorial · v1.4

Revision history

Version	Changes
V1.0	Initial release, 4 July 2020.
V1.1	Added the VGA colour-bar display chapter; the OV7670-to-VGA display chapter; and the dual-camera MT9V011 USB 2.
V1.2	Added the dual-camera MT9V011 VGA display chapter (24 October 2021).
V1.3	Added USB Slave FIFO read and 2.4-inch LCD colour-bar display chapters (7 October 2022).
V1.4	Added OV7670-to-2.4-inch-LCD display and USB-to-2.4-inch-LCD image display chapters (9 October 2022).

Contents

Verilog HDL Digital System Design

1. Board overview
2. Software installation — Quartus II and USB-Blaster
3. Combinational logic gate modelling
4. 2-to-4 decoder
5. D latch, D flip-flop and sequential logic
6. LED blinking
7. Push-button detection and debouncing
8. PLL
9. Seven-/eight-segment stopwatch
10. VGA colour-bar display
11. Digital frequency counter and SignalTap II
12. OV7670 camera display on a VGA monitor
13. USB Slave FIFO write (FPGA to PC)
14. USB Slave FIFO read (PC to FPGA)
15. USB video display on the host PC
16. OV7670 video display on the host PC over USB
17. 2.4-inch LCD colour-bar display
18. OV7670 camera display on a 2.4-inch LCD
19. USB image display on a 2.4-inch LCD
20. USB image display on VGA

Verilog HDL Digital System Design

This part of the tutorial introduces practical FPGA digital-system design using Verilog HDL on the ETree FCore2S development board. The examples progress from basic logic gates and sequential circuits to display interfaces, camera capture, SDRAM buffering and USB 2.0 communication.

Chapter 1. Board overview

The ETree FCore2S is a core board combining an FPGA with a USB 2.0 interface implemented by the Cypress CY7C68013A. Its resources are:

• FPGA: EP4CE6E22C8N or EP4CE10E22C8N

• SDRAM: W9825G6KH-6

• SPI Flash: W25X40

• USB 2.0: CY7C68013A

• EEPROM: 24LC64

• LED

• Independent push-button

• JTAG programming interface

• GPIO interface

The board is offered through the ETree Taobao store: https://shop397545458.taobao.com.

Chapter 2. Software installation

2.1 Quartus II

Quartus II is Altera's programmable-logic design environment. The tutorial uses Quartus II version 13.1.0.162. The author refers readers to the accompanying installation video for the complete procedure.

• Run QuartusSetup-13.1.0.162.exe from the software directory.

• Proceed through the installer using Next and accept the licence agreement.

• Choose the installation directory (the default is the root of drive C:).

• Install the required 32-bit version and also select the 64-bit version. Select support for Cyclone III/IV and MAX II/V devices.

• Complete installation, then specify the location of the licence file when prompted.

• Select the directory containing license.dat. The source document reports that its licence setup displays an expiry date of December 2035.

2.2 USB-Blaster driver installation

Before using Quartus II to program or debug the development board, install the USB-Blaster driver. The driver is located on the supplied media under Tools\usb-blaster. In Windows Device Manager, select the corresponding device and proceed with driver installation. When the security prompt appears, choose “Always trust” and click Install. When installation finishes, close the dialog and verify that USB-Blaster appears in Device Manager.

Chapter 3. Combinational logic gate modelling

This chapter introduces the basics of modelling combinational logic gates in Verilog.

3.1 Gate-level modelling of combinational circuits

Gate-level modelling describes a logic circuit diagram using the constructs defined by Verilog HDL. Verilog provides twelve basic gate primitives:

• and — multi-input AND gate

• or — multi-input OR gate

• xor — multi-input exclusive-OR gate

• buf — one-input, multiple-output buffer

• nand — multi-input NAND gate

• nor — multi-input NOR gate

• xnor — multi-input XNOR gate

• not — one-input, multiple-output inverter

• bufif1 — tri-state buffer enabled by a logic-high control signal

• bufif0 — tri-state buffer enabled by a logic-low control signal

• notif1 — tri-state inverting buffer enabled by a logic-high control signal

• notif0 — tri-state inverting buffer enabled by a logic-low control signal

3.2 Multi-input gates

A multi-input gate has one output. The tutorial illustrates an AND gate and its Verilog instance syntax: `and a1(out,in1,in2,in3);`.

3.3 Multiple-output gates

A multiple-output gate has one input. The tutorial illustrates an inverter with the syntax: `not n1(out1,out2,...,in);`.

3.4 Tri-state gates

A tri-state gate has one data input, one output and one control input. When the control signal is inactive, the output is high impedance (Z). The example is `bufif1 b1(out,in,ctrl);`.

Chapter 4. 2-to-4 decoder

This chapter uses the basic Verilog logic-gate primitives to implement a 2-line-to-4-line decoder.

4.1 Verilog source-code structure

Before designing the decoder, the tutorial introduces the structure of Verilog source code. A top-level module organizes the relationships between modules. It instantiates the required submodules by module name and port name, and maps the signals between them. Other modules can be instantiated in the same way.

4.2 Gate-level modelling

Assume the two inputs are A1 and A0 and the four outputs are Y[3:0]. The decoder truth table is:

A1, A0	Y[3:0]
0, 0	0001
0, 1	0010
1, 0	0100
1, 1	1000

The gate-level implementation inverts A1 and A0, then uses four AND gates to generate the four one-hot output combinations.

4.3 Dataflow modelling

The same decoder can be implemented using dataflow modelling. The input and output interface remains unchanged; continuous assign statements express the combinational function directly. The tutorial notes that this approach is much shorter and is the method most frequently used in Verilog design.

Chapter 5. D latch, D flip-flop and sequential logic

5.1 D latch and D flip-flop

Behavioural modelling describes the function or algorithm of a logic circuit and is commonly implemented with an always block. The D latch example is sensitive to changes on E or D; when E is high, Q follows D. The D flip-flop example uses `always @(posedge clk)` and transfers D to Q on the rising edge of clk.

5.2 Sequential logic: decade counter

The main difference between sequential and combinational logic is that sequential logic is clock-driven: data is processed in step with the clock. The example is a decade counter with a 4-bit output Q[3:0], because the largest decimal digit is 9 (binary 1001). On each rising edge of clk, Q increments by one until it reaches 9, then returns to zero.

Chapter 6. LED blinking

This is the first FPGA project and explains the process of creating a Quartus II project. Readers already familiar with the Quartus II workflow may skip this chapter or use it as a reference. The objective is to make the LED blink once per second.

6.1 Hardware

The development board has two LEDs: one power indicator and one LED controlled by the FPGA. The FPGA-controlled LED is active high.

6.2 Create a project

Start Quartus II and select File → New Project Wizard. Enter top_fpga as both the project name and the top-level design name. Select the device family and part number, then choose ModelSim/Verilog as the simulation tool and language. Finish the wizard.

6.3 HDL design

Create top_fpga.v using File → New → Verilog HDL File. The example uses a 24 MHz input clock, an active-low reset and an active-high LED output. A 32-bit timer is cleared by reset and counts clock cycles. The LED-control logic turns the LED on at the stated timer count and off at the one-second count. Save the source as top_fpga.v.

6.4 Dual-Purpose Pins

Configure the Dual-Purpose Pins settings as shown in the original screenshots.

6.5 Compile and assign pins

Run Analysis & Synthesis. Open Assignment → Pin Planner and add the pin assignments. Set Unused Pins to input tri-state, then run Compile Design to compile the complete project.

6.6 Program the SOF file

The .sof file is normally used to verify a design and debug the FPGA online. It configures the FPGA directly, but the configuration is lost when power is removed. Open Program Device (Open Programmer), select USB-Blaster [USB-0] in Hardware Setup and choose JTAG mode. Select top_fpga.sof from output_files and click Start. A successful download is indicated by a green progress bar reaching 100%, after which the LED should blink.

6.7 Generate a JIC file

After verifying the design, the compiled configuration can be stored in external flash so that the flash automatically configures the FPGA at power-up. A JIC file is used to program the EPCS4 configuration device (SPI Flash), preserving the FPGA configuration across power cycles. Select File → Convert Programming File, choose Cyclone IV as the device family and EP4CE10 as the device name, then generate output.jic.

6.8 Program the JIC file

Remain in JTAG mode, add the output_file.jic file, enable Program/Configure and click Start. After successful programming, the LED should continue blinking after the board is power-cycled.

Chapter 7. Push-button detection

Building on the previous chapter, this project adds a pause/resume function. Pressing the button stops the LED blinking; pressing it again resumes blinking.

7.1 Hardware

The circuit uses push-button S1. It is active low: when pressed, KEY0 is connected to GND.

7.2 HDL design and debouncing

Because of mechanical and operating characteristics, a push-button typically bounces for about 20 ms when pressed or released. During this interval its logic level changes repeatedly, making it difficult for the FPGA to detect a clean transition. The tutorial removes this bounce.

The simple method described is to repeatedly sample the corresponding pin after a button press. If the pin remains low for more than 20 ms, the press is accepted as valid. The chapter provides a key_jitter.v debouncing module.

7.3 Pin assignment

Open Assignment → Pin Planner and add the required pin constraints.

7.4 Experiment

After downloading the FPGA configuration, the LED initially blinks once per second. Pressing the button below the board stops the LED in its current state. Pressing it again resumes the one-second blinking cycle. This behaviour repeats.

Chapter 8. PLL

The previous chapters did not use the IP components built into Quartus II, known as MegaWizard. This chapter demonstrates creating a PLL using MegaWizard.

8.1 Hardware

The hardware platform is the same as the LED-blinking project in Chapter 6.

8.2 MegaWizard

In Quartus II, open Tools → MegaWizard Plug-In Manager and choose to create a new megafunction variation. The left-hand panel lists nine IP categories. Arithmetic includes basic addition, subtraction, multiplication, division and square-root operations. DSP includes FIR, FFT and digital image-processing functions. I/O includes DDIO, LVDS and ALTPLL. Interface includes interfaces such as DDR, DDR2 and PCI. Memory Compiler includes memory structures such as FIFO and dual-port RAM.

Select ALTPLL under I/O and name the instance my_pll. Set the input clock to 24 MHz. Clear the option to generate areset. Under Output Clocks, set the clock division factor to 240, phase to 0 degrees and duty cycle to 50%, producing the required 100 kHz clock. In Summary, select my_pll_inst.v to generate the instantiation file, then click Finish.

8.3 HDL design

The top_fpga.v example instantiates my_pll, takes the 24 MHz input clock and obtains clk_100k from the PLL. The timer counts to 99,999, corresponding to one second at 100 kHz; the LED is switched at the half-second and one-second count values.

8.4 Experiment

After downloading the configuration, the LED blinks once per second, equivalent to the Chapter 6 result. This demonstrates that the PLL is operating.

Chapter 9. Seven-/eight-segment stopwatch

This chapter implements a stopwatch using a two-digit seven-/eight-segment display. The external VGA_SM module board is required.

9.1 Hardware

LEDs are commonly used as indicators in electronic products. Another common display device is the seven-segment display, used in products such as electronic clocks and multimeters. A display digit is essentially an arrangement of LEDs. The most common types are seven-segment and eight-segment displays; the eight-segment version adds a decimal-point LED, usually at the lower right.

The illustrated display is common-anode: the anodes of all eight LEDs share a common connection. A segment lights when its cathode is driven low and turns off when its cathode is high; this is described as active-low.

The parameter encodings correspond, from the most significant to the least significant bit, to {a, b, c, d, e, f, g, dp}.

9.2 HDL design

The system consists of a seconds timer, stopwatch carry counter, digit-selection counter, digit decoder and display logic. The units digit second_0 increments from 0 through 9 and then returns to zero. When second_0 reaches 9, it generates a carry to second_1. The tens digit second_1 increments up to 5 and then returns to zero when the units digit rolls over, implementing a 00–59 seconds count.

9.3 Pin assignment

Open Assignment → Pin Planner and add the pin constraints.

9.4 Experiment

Insert the VGA_SM module board, ensuring that pin 1 aligns with pin 1 of P2, and connect the Mini-USB cable. Download the FPGA configuration; the seven-segment display should begin counting seconds.

Chapter 10. VGA colour-bar display

This chapter uses the VGA interface to display colour bars. The external VGA_SM module board is required.

10.1 Hardware

The FPGA board implements a VGA565 interface using a parallel resistor voltage-divider network: RED has 5 bits, GREEN has 6 bits and BLUE has 5 bits. This provides 65,536 possible colours. To reduce cost, the design uses a resistor network as a simple DAC instead of an ADV7123 video DAC. The source and termination matching resistors for each of the three colour channels are 75 ohms.

The VGA interface specifies an analogue voltage range of 0 to 0.714 V for each primary colour. Different resistor-divider networks generate the required voltage levels.

10.2 VGA timing

VGA (Video Graphics Array) was introduced by IBM in 1987. It offers high resolution, a high refresh rate and a wide range of colours, and became widely used for colour displays. VGA does not support hot plugging or audio transmission.

This design uses 640 × 480 at 60 Hz. The pixel clock is calculated as 800 × 525 × 60 = 25,200,000 Hz. HSYNC (horizontal sync) and VSYNC (vertical sync) are both active low. HSYNC must also be generated during the horizontal blanking interval.

10.3 HDL design

The system framework contains a my_pll module, which generates the required clock, and a VGA interface module, which generates the VGA timing signals. The vga_show module generates the VGA output. Its parameters define the horizontal line period (800 clocks), horizontal sync pulse (96), back porch (48), active width (640) and front porch (16), together with the vertical frame period (525 lines), vertical sync pulse (2), back porch (33), active height (480) and front porch (10). The design generates red, green, blue and yellow vertical bars.

10.4 Pin assignment

Open Assignment → Pin Planner and add the pin constraints.

10.5 Experiment

Insert the VGA module board, ensuring that pin 1 aligns with pin 1 of P2, and connect the FPGA board to a VGA monitor. Power on the board and download the FPGA configuration. The monitor should display red, green, blue and yellow colour bars.

Chapter 11. Digital frequency counter

This chapter implements a digital frequency counter with a range of 0–255 Hz. It measures the default frame rate of the OV7670 camera and uses the embedded logic analyser SignalTap II to observe the result.

11.1 Hardware

Connect the camera module to P3. The signal under measurement is CMOS_FV (VSYNC) on the OV7670 camera interface, corresponding to pin EXT2_3 of connector P3.

11.2 HDL design

The 24 MHz oscillator on the board is used as the timing reference. The design defines a one-second counter and a frequency counter. It counts the rising edges of the CMOS_FV frame-sync signal during each one-second interval to calculate the CMOS image sensor's output frame rate.

11.3 Pin constraints

Open Assignment → Pin Planner and add the pin constraints.

11.4 Using SignalTap

• Open SignalTap II Logic Analyzer.

• Select the JTAG Chain Configuration.

• Set the sampling clock and sample depth. First select the sampling clock; choose the pre-synthesis filter; display the matching signals; select clk and add it.

• Complete the sampling-clock selection, set the sample depth to 128 and choose Continuous triggering.

• Add the signals to be observed by double-clicking the blank area in the Setup pane. The example selects dat_out.

• Save the SignalTap II file. Quartus II asks whether to enable SignalTap II; click Yes. To disable it later, change the setting in Settings.

• Select the signal trigger condition.

• Recompile the project whenever a SignalTap II file is added or modified.

• Run SignalTap II.

11.5 Experiment

Connect the JTAG programmer, MINI-USB power cable, OV7670 camera module (pin 1 to pin 1 of P3) and VGA module board (pin 1 to pin 1 of P2). Download top_fpga.sof, open SignalTap II and click the single-trigger button. The measured value is initially shown as 1Eh (hexadecimal). To make it easier to read, right-click frame_out and select unsigned decimal display. Trigger again: the measured frequency should be 30. According to the OV7670 datasheet, the actual output frame rate is also 30 frames per second.

Chapter 12. OV7670 camera display on a VGA monitor

This chapter uses an OV7670 camera and the VGA interface to display camera data in real time on a VGA monitor.

12.1 Hardware

The design uses an OV7670 camera module and a VGA_SM module. The original document provides the OV7670 hardware schematic.

The OV7670 output interface is a parallel DVP signal. The sensor provides approximately 300,000 pixels and offers a high cost/performance ratio.

12.2 OV7670 camera

The OV7670 is a 1/6-inch CMOS VGA image sensor manufactured by OmniVision. It is small and operates at a low voltage, integrating the functions of a VGA camera and image processor. It is controlled through the SCCB bus and can output full-frame, subsampled or windowed 8-bit image data at various resolutions. Its maximum VGA frame rate is 30 frames per second.

The user can control image quality, data format and transmission method. Image-processing functions such as gamma correction, white balance, brightness and chroma can be programmed through SCCB. OmniVision's sensor technology reduces or eliminates optical and electronic defects such as fixed-pattern noise, trailing and blooming, improving image clarity and stability.

The listed features are: high sensitivity and low operating voltage for embedded applications; a standard SCCB interface compatible with I²C; RAW8, RGB (including GBR 4:2:2, RGB565, RGB555 and RGB444), YUV 4:2:2 and YCbCr 4:2:2 output formats; VGA, CIF and smaller output sizes down to 40 × 30; automatic exposure, gain, white balance, banding-filter and black-level calibration; adjustable saturation, hue,

gamma and sharpness; flash support; and image scaling.

The OV7670 registers are accessed and configured using SCCB timing, which is very similar to I²C. The source recommends consulting the relevant documentation for details. Its output signals are: PCLK (pixel clock), VSYNC (frame sync), HREF/HSYNC (line sync) and D[7:0] (image data). When HREF is high, one byte is output on each PCLK cycle. A high VSYNC indicates the start of a new frame.

12.3 CMOS operating principle

APS (Active Pixel Sensor) is a CMOS image-sensor pixel architecture. The source illustrates a 3T-APS circuit and describes its operation as follows:

• Turn on reset transistor Mrst.

• Reset the photodiode (PD) to Vrst − Vth, where Vth is the threshold voltage of transistor Mrs.

• Turn off Mrst so that the PD enters a floating electrical state.

• Incident light causes photogenerated carriers to accumulate in the PD junction capacitance.

• After an integration period, enable transistor Msf.

• Turn on Msel; the pixel output signal is read on the vertical output line.

• Turn off Msel, turn Mrst on again and repeat the process.

The document then describes the MT9V011 as a 300,000-pixel sensor capable of up to 30 frames per second, with a 1/3-inch photosensitive array. APS Array and Analog Processing are the analogue sections; the photosensitive array uses a GRBG arrangement. Control Register and Timing and Control are digital sections. Serial IO is the I²C interface, and Data Out is the parallel DVP video interface. The DVP waveform uses LINE_VALID (HREF), PIXCLK, FRAME_VALID and Dout.

12.4 HDL design

The FPGA stores captured OV7670 data in SDRAM. The VGA driver then reads the video data from SDRAM and sends it to the VGA monitor. The system modules are:

• CMOS driver — captures and crops the CMOS image.

• RAW2RGB — converts raw data to RGB. The text says this conversion is performed inside the camera chip, so the FPGA receives RGB565 data directly.

• FIFO write — transfers RGB data to the SDRAM controller.

• FIFO read — reads video-frame data from SDRAM and supplies the VGA driver.

• VGA module — sends the video signal to the VGA monitor.

The VGA driver example uses a 25.175 MHz clock and 640 × 480 active video. The SDRAM driver is modified from the open-source SDRAM controller supplied by Terasic; the source does not explain its implementation.

12.5 Pin constraints

Open Assignment → Pin Planner and add the pin constraints.

12.6 Experiment

Connect the JTAG programmer, MINI-USB power cable, OV7670 camera module (pin 1 to pin 1 of P3) and VGA module board (pin 1 to pin 1 of P2). Download the compiled top_fpga.sof file. The resulting camera image is shown in the original document.

Chapter 13. USB Slave FIFO write

This chapter uses the FPGA and a USB 2.0 CY7C68013A device to send data packets from the FPGA to a PC.

13.1 Hardware

The development board has an onboard USB 2.0 chip. The original document includes its circuit schematic.

13.2 CY7C68013A overview

The CY7C68013A is a high-speed USB 2.0 device from Cypress. It integrates a microprocessor for the USB 2.0 protocol and supports transfer rates up to 480 Mb/s. It supports control, interrupt, bulk and isochronous transfers.

Its architecture includes a Serial Interface Engine (SIE), which handles USB-related functions such as serial-data encoding/decoding, error control and bit stuffing. The embedded enhanced 8051 MCU is responsible for register configuration and control.

13.3 USB Slave FIFO transfers

Slave FIFO mode can be used when an external controller such as an FPGA needs high-speed communication with a host through the FX2 (CY7C68013A), and the external controller can generate the timing required by the Slave FIFO interface. In this mode, the embedded 8051 firmware only configures the Slave FIFO registers and controls when the FX2 operates in Slave FIFO mode.

The interface signals are:

• IFCLK — clock output by the FX2; it can be used as the synchronous communication clock.

• FLAGA/FLAGB/FLAGC/FLAGD — FIFO status outputs from the FX2, indicating conditions such as full or empty.

• SLCS — FIFO chip select, controlled by external logic. When high, data transfer is disabled.

• SLOE — FIFO output enable, controlled by external logic. When inactive, the data bus does not output valid data.

• SLRD — FIFO read control. In synchronous-read mode, the FIFO pointer advances on each IFCLK rising edge while SLRD is active. In asynchronous-read mode, it advances on each active-to-inactive transition of SLRD.

• SLWR — FIFO write control. In synchronous-write mode, data is written and the FIFO pointer advances on each IFCLK rising edge while SLWR is active. In asynchronous-write mode, data is written on each active-to-inactive transition of SLWR.

• PKTEND — packet-end signal. Normally, when the number of bytes written reaches the packet size configured in FX2 firmware, the data is automatically packetised. If the external logic needs to send a packet smaller than that configured size, it asserts PKTEND after writing the desired number of bytes; the FX2 then commits those bytes as a packet.

• FD[15:0] — data bus.

• FIFOADR[1:0] — selects one of the four FIFO endpoint addresses and is controlled by the external logic.

The example uses synchronous FIFO read and write, with IFCLK supplied by the FPGA.

13.4 USB firmware development

USB 2.0 firmware is developed in Keil. Add the EZUSB object file USBJmp.OBJ and library EZUSB.LIB to the project. The three important source files are:

• fw.c (firmware) — handles USB protocol communication, including power-on enumeration, re-enumeration, wake-up and calling user code. This file normally does not need modification.

• bulkloop.c — contains user code; Cypress supplies the framework. It includes TD_Init(), TD_Poll(), DR_VendorCmnd(), and endpoint interrupt-service routines.

• dscr.51 — contains the USB descriptors, including device, interface, endpoint and string descriptors. Modify it as required.

The firmware exposes two endpoints: EP6 is used for PC-to-USB-chip transfers (OUT), and EP2 is used for USB-chip-to-PC transfers (IN). The endpoint descriptors are defined in dscr.51.

In TD_Init(), configure the Slave FIFO clock to be supplied externally by the FPGA. The example uses an external 48 MHz clock (IFCONFIG = 0x03). Configure the data interface as 8-bit, with WORDWIDE = 0. In DR_VendorCmnd(), define VR_NAKALL_ON (0xA8) and VR_NAKALL_OFF (0xA9) to handle commands sent by the host. The example uses NAK-ALL and FIFO reset operations to avoid race conditions. For the remaining firmware details, consult the project source and the official technical documentation.

13.5 Set up a Keil project

Run Setup.exe in USB2.0\uV2_4k\setup and follow the installer screens shown in the original document.

13.6 HDL design

According to the USB 2.0 Slave FIFO interface definition, the FPGA writes test data whenever the USB chip's Slave FIFO is not full. The test data increments by one on each valid clock cycle. The example module is usb_ctrl.v.

13.7 Pin constraints

The pin assignments are shown in the original screenshots.

13.8 Experiment

Install the USB 2.0 driver, download the USB firmware, and program the FPGA configuration. For the host-side demonstration, reconnect the MINI-USB 2.0 cable (power-cycle the board), then run CyControl.exe from USB2.0\USB Control Center. Select Bulk IN endpoint (0x82), set the receive length to 512 and click Transfer Data-IN. The host should display the incrementing test data sent by the FPGA.

Chapter 14. USB Slave FIFO read

The previous chapter sent data packets from the FPGA to the PC. This chapter adds the reverse direction: the PC sends data packets to the FPGA.

14.1 Hardware

The development board has an onboard USB 2.0 chip; see the schematic in the original document.

14.2 USB firmware

The USB firmware development is the same as in the previous chapter.

14.3 HDL design

According to the USB 2.0 Slave FIFO interface definition, the FPGA reads test data when the USB chip's Slave FIFO is not empty. The example module is usb_ctrl.v and implements the USB Slave FIFO read function.

14.4 Pin constraints

The pin constraints are the same as in the previous chapter.

14.5 Experiment

Download the FPGA configuration and USB firmware. Open SignalTap II and configure a rising-edge trigger on ext_data_valid, then wait for the host to send data. Run CyControl.exe from USB2.0\USB Control Center, select Bulk OUT endpoint (0x06), enter eight bytes (0x12, 0x34, 0x56, 0x78, 0x12, 0x34, 0x56, 0x78) and click Transfer. SignalTap II should capture the eight bytes sent by the PC, matching the transmitted values.

Chapter 15. USB video display on the host PC

The FPGA generates a video signal at 640 × 480 and 30 frames per second and sends it to the USB 2.0 CY7C68013A. The USB chip packetises the video data and sends it to the PC. Host software uses the USB driver to capture and play the video.

15.1 Hardware

The development board has an onboard USB 2.0 chip; see the schematic in the original document.

15.2 USB firmware

The USB firmware is the same as in the previous chapter.

15.3 HDL design

The system framework has two main modules. vid_test generates test video data at 640 × 480 and 30 frames per second. usb_ctrl configures the USB 2.0 chip pins and forwards data to its Slave FIFO interface. The test video consists of incrementing data that repeats cyclically.

15.4 Pin constraints

Open Assignment → Pin Planner and add the pin constraints.

15.5 Experiment

Download the FPGA configuration and USB firmware, then reconnect the MINI-USB 2.0 cable to power-cycle the board. Open USB_DEMO in the USB2.0 directory and click Play. The original document shows the resulting video display.

Chapter 16. OV7670 USB display on the host PC

This chapter sends OV7670 camera video data to a PC over USB for display at 30 frames per second.

16.1 Hardware

The hardware was introduced in previous chapters and is not repeated here.

16.2 HDL design

The FPGA stores the captured OV7670 camera data in SDRAM. A USB driver module then transfers the video data from SDRAM to the USB 2.0 interface chip, allowing the host PC to display it in real time. The source refers readers to the project for the detailed code.

16.3 Pin constraints

Open Assignment → Pin Planner and add the pin constraints.

16.4 Experiment

Connect the JTAG programmer, MINI-USB power cable and OV7670 camera module (pin 1 to pin 1 of P3). Program output_file.jic into the SPI Flash and power-cycle the board. Open USB_Demo in the Tools directory and click Play to view the live video.

Chapter 17. 2.4-inch LCD colour-bar display

This chapter implements a colour-bar display on a 2.4-inch LCD module.

17.1 Hardware

The TFT LCD module has a resolution of 320 × 240 and uses the ILI9341 controller. It has an MCU parallel interface and can display 16-bit true colour. The ILI9341 controller includes its own display RAM, with a capacity of 172,800 bits (240 × 320 × 18 bits). In 16-bit mode, pixel data is stored in RGB565 format. The original document shows the relationship between the controller's 18 data lines and the LCD GRAM.

17.2 TFT-LCD overview

A TFT-LCD communicates with its host controller through an MCU parallel (8080-style) bus. This bus is widely used for LCD modules and includes the following signals:

• CS — chip select.

• WR — write control.

• RD — read control.

• DATA[15:0] — bidirectional data bus.

• RST — reset.

• DC — command/data indicator (0: read in the source's description; 1: write).

For an MCU-bus read or write, first set DC according to the type of transfer, then assert CS low and drive RD or WR low. Data is read on the rising edge of RD and written on the rising edge of WR.

The tutorial says that the LCD is driven by emulating the MCU bus with ordinary STM32 GPIO pins. PB0–PB15 provide the 16-bit data bus to improve transfer speed.

Important ILI9341 commands include 0x36 (Memory Access Control), which controls read/write and scan direction through parameters such as MY, MX and MV; 0x2A (column-address set), which sets the horizontal coordinate x using start and end values SC and EC; 0x2B (page-address set), which sets the vertical coordinate y using SP and EP; 0x2C (write GRAM), after which colour data can be written continuously; and 0x2E (read GRAM), which reads the controller's display memory.

17.3 HDL design

The design only needs to implement parallel writes to the 2.8-inch LCD mentioned in the source, displaying red, green, blue and yellow bars. The LCD uses 16-bit RGB565. The colour constants are RED = 16'hf800, GREEN = 16'h07e0, BLUE = 16'h001f and YELLOW = 16'hffe0.

The tutorial does not explain the LCD register configuration in detail; it refers readers to the ILI9341 datasheet. It provides a lookup table (lut_data) containing the controller initialisation commands and parameters, including power control, VCM control, memory access control, pixel format, gamma settings, sleep exit, display-on, X/Y address ranges and the GRAM write command.

17.4 Pin assignment

Open Assignment → Pin Planner and add the pin constraints.

17.5 Experiment

Insert the 2.4-inch LCD module into connector P2, observing the pin-1 orientation. Connect the USB-Blaster programmer and USB power cable, then download the FPGA configuration. The original document shows the resulting display.

Chapter 18. OV7670 camera display on a 2.4-inch LCD

This chapter uses an OV7670 camera module and a 2.4-inch LCD to display camera data in real time.

18.1 Hardware

The source marks this section as omitted.

18.2 HDL design

The system framework contains the following modules:

• CMOS driver — captures and crops the CMOS image.

• RAW2RGB — converts raw data to RGB. The source says this conversion is performed inside the camera chip, so the FPGA receives RGB565 data directly.

• FIFO write — transfers RGB data to the SDRAM controller.

• FIFO read — reads video-frame data from SDRAM and supplies the LCD driver.

• LCD module — sends the video signal to the external LCD for display.

The SDRAM driver is modified from the open-source controller supplied by Terasic. It implements SDRAM read and write control; the tutorial does not explain it further.

18.3 Pin constraints

Open Assignment → Pin Planner and add the pin constraints.

18.4 Experiment

Insert the 2.4-inch LCD module into P2 and the OV7670 camera into P3, observing the pin-1 orientation for both. Connect the USB-Blaster programmer and USB power cable, then download the FPGA configuration. The original document shows the resulting camera display.

Chapter 19. USB image display on a 2.4-inch LCD

The previous chapter displayed camera data on a 2.4-inch LCD. This chapter displays image data sent by the host PC through the USB 2.0 chip.

19.1 Hardware

The source marks this section as omitted.

19.2 HDL design

The system framework contains the following modules:

• USB driver — receives data sent through the CY7C68013A; see the USB Slave FIFO read chapter for the principle.

• RAW2RGB — converts raw image data to RGB. In this project, the Img2Lcd.exe image-conversion software performs this function, so the FPGA receives RGB565 data directly.

• FIFO write — transfers RGB data to the SDRAM controller.

• FIFO read — reads video-frame data from SDRAM and supplies the LCD driver.

• LCD module — sends the video signal to the external LCD for display.

The SDRAM driver is modified from the open-source controller supplied by Terasic. It implements SDRAM read and write control; the tutorial does not explain it further.

19.3 Pin assignment

Open Assignment → Pin Planner and add the pin constraints.

19.4 Experiment

Before the experiment, generate a binary file for the image. Run Img2Lcd.exe from Tools/Image2Lcd 2.9. Set the output format to binary (.bin), horizontal scan order, 16-bit true colour, and maximum width and height of 320 and 240.

Open basket.bmp in the “Test Images” directory and save the converted binary data as basket.bin. Insert the 2.4-inch LCD module into P2, connect the USB-Blaster programmer and USB power cable, then download the FPGA configuration.

Run CyControl.exe from USB2.0\USB Control Center. Select Bulk OUT endpoint (0x06), click Transfer and select the basket.bin file generated above. The image should then be displayed on the LCD.

Chapter 20. USB image display on VGA

The table of contents lists a final chapter titled “USB Image VGA Display” beginning on page 127. The extracted source text contains no further explanatory content for this chapter.

End of translated text

This English document is a companion translation of the extractable Chinese text in ETree's FCore2S tutorial, version 1.4. It is not a replacement for the original diagrams, screenshots or HDL source listings.